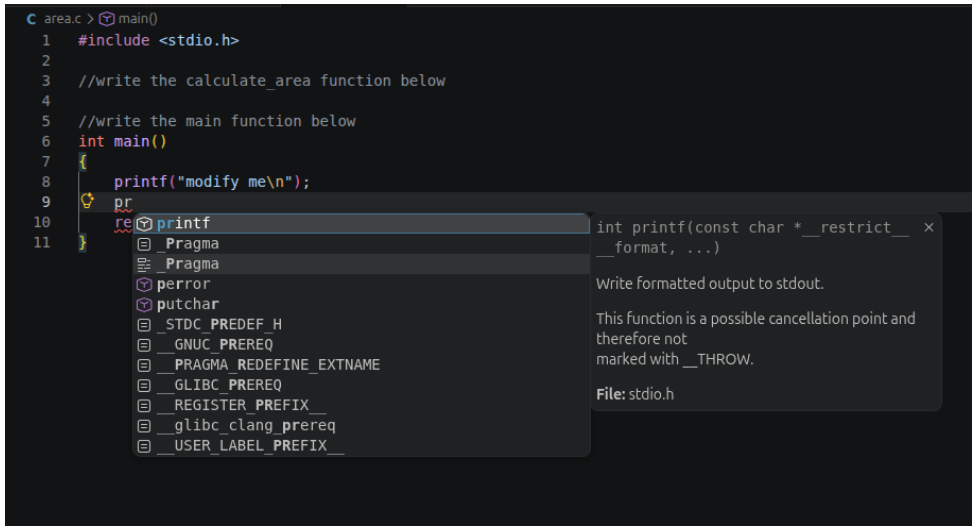
Lab 4: Arrays and Pointers

Matteo Fasiolo

VSCode set up (ignore if you have already done this)

VScode comes with auto-suggestion features:



These are useful for experienced programmers, but are detrimental for learners.

The exam is pencil-and-paper, so it's best to disable all these features.

The next slide explains how.

Open VSCode and press Ctrl+Shift+P (Cmd+Shift+P on Macs).

In the text box, type `Preferences: Open User Settings (JSON)`.

Copy-paste this code (including brackets) into that file:

```
{
"chat.disableAIFeatures": true,
"editor.inlineSuggest.enabled": false,
"editor.quickSuggestions": false,
"editor.suggestOnTriggerCharacters": false,
"editor.wordBasedSuggestions": "off",
"editor.parameterHints.enabled": false,
"editor.snippetSuggestions": "none",
"editor.tabCompletion": "off",
"editor.acceptSuggestionOnEnter": "off",
"editor.acceptSuggestionOnCommitCharacter": false,
"C_Cpp.autocomplete": "disabled"
}
```

Homework 1: working with arrays

Open the file `arrays.c` and complete the following tasks.

A) Complete the function

```c
double maximum(const double values[], int length);
```

The function should:

1. initially treat `values[0]` as the largest value;
2. inspect the remaining elements;
3. return the largest value.

Test it in `main()` using the array provided in the file.

B) Complete the function

```
void swap_first_two(double values[], int length);
```

- If `length >= 2`, swap `values[0]` and `values[1]`.
- Otherwise, leave the array unchanged.

Test it in `main()` using the array provided in the file.

C) Given the matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

stored in a row-major integer array, complete the function

```
void print_matrix(const int values[], int nrow,
                  int ncol);
```

and test that it prints the test matrix in `arrays.c` correctly.

Homework 2: working with pointers

Open the file `pointers.c`.

A) Complete the function

```
int range(const double values[], int length,
          double *minp, double *maxp);
```

The function should find the smallest and largest elements of `values` and store the results at the addresses supplied through `minp` and `maxp`.

It should return `1` after successfully producing both results.

The function should return `0` without accessing the array when:

- `length <= 0`;
- `values == NULL` (that is, when `values` is a NULL pointer);
- `minp == NULL` or `maxp == NULL`.

Test the function using the array provided in `main()`.

While stopped inside `range`, use the debugger to confirm that its pointer parameters `minp` and `maxp` refer to the variables `minimum` and `maximum` declared in main.

B) In the same file, complete

```
void swap_min_max(double values[], int length);
```

The function should:

1. scan the elements of `values` using a pointer;
2. use two further pointers (`minp` and `maxp`) to remember the locations of the smallest and largest elements;
3. swap the selected elements by dereferencing the pointers `minp` and `maxp`.

If `values == NULL` or `length < 2`, leave the array unchanged.

If the minimum or maximum occurs more than once, use its first occurrence.

For the array supplied in `main()`, the result should be

```
Before: 4.20 -1.00 3.80 7.10 2.40
After:  4.20 7.10 3.80 -1.00 2.40
```

Immediately before the swap, use the debugger to inspect `minp`, `maxp`, `*minp` and `*maxp`.

Optional challenge: sorting

Sorting means rearranging a collection of values into a chosen order.

Here we want to sort numbers into **ascending order**, from smallest to largest. For example,

```
Before:  4.20 -1.00 3.80 7.10 2.40
Sorted: -1.00  2.40 3.80 4.20 7.10
```

In a sorted array, every element is less than or equal to the element that follows it.

Bubble sort achieves this by repeatedly scanning neighbouring elements. Whenever the left element is larger than the right element, it swaps them.

Optional challenge: bubble sort

The pseudocode is:

```
repeat length - 1 times:
    for i from 0 to length - 2:
        if values[i] > values[i + 1]:
            swap values[i] and values[i + 1]
```

In `pointers.c`, complete

```
void swap(double *left, double *right);
void bubble_sort(double values[], int length);
```

Perform each swap by passing the addresses of the two elements:

```
swap(&values[i], &values[i + 1]);
```

Submission

Upload your solutions to homeworks 1 and 2 in two separate `.c` files.

NOTE Blackboard allows uploading multiple files, so this should work fine.

Before submitting, make sure your code compiles and runs!